🧠 AI Evolution Program

Parte 08 — Recuperación, contexto, memoria y conocimiento

Construye sistemas que conectan modelos con conocimiento verificable mediante búsqueda, RAG, grafos, memoria y evaluación de atribución.

1. 100 — Embeddings y búsqueda vectorial
2. 101 — Segmentación, metadatos y ventanas
3. 102 — Búsqueda léxica y BM25
4. 103 — Búsqueda híbrida y fusión de rankings
5. 104 — Re-ranking y filtros de evidencia
6. 105 — RAG básico con citas
7. 106 — Transformación y descomposición de consultas
8. 107 — Knowledge graphs y GraphRAG
9. 108 — Memoria de corto y largo plazo
10. 109 — Compresión de contexto y cachés semánticos
11. 110 — Evaluación de fidelidad, cobertura y atribución
12. 111 — Proyecto: RAG productivo y auditable

100 — Embeddings y búsqueda vectorial

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **embeddings y búsqueda vectorial** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar embeddings y búsqueda vectorial usando los conceptos `embeddings`, `vector search`, `cosine`, `índices`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`embeddings`, `vector search`, `cosine`, `índices`

🗺️ Ubicación en el mapa de la IA

Los embeddings son el puente entre el texto discreto y la geometría continua: convierten palabras, frases y documentos en vectores densos donde la cercanía codifica similitud semántica. Heredan la hipótesis distribucional de la lingüística de corpus y los modelos de lenguaje neuronales (partes 05 y 06), y son el cimiento de toda la parte 08: sin búsqueda vectorial no hay RAG, ni memoria a largo plazo, ni caches semánticos. Lo que esta clase establece —representar para recuperar— reaparece en cada clase siguiente.

📖 Fundamentos

🔢 Qué es un embedding

Un **embedding** es una función `E: texto → ℝ^d` aprendida por una red neuronal, con `d` típicamente entre 256 y 3072. La propiedad que la hace útil es que la geometría refleja la semántica: textos con significado parecido quedan en regiones cercanas del espacio, aunque no compartan ninguna palabra ("automóvil" y "coche" acaban próximos). Los modelos modernos (p. ej. Sentence-BERT y sus derivados) se entrenan con

aprendizaje contrastivo: acercar pares relacionados (pregunta/respuesta, oración/paráfrasis) y alejar pares negativos.

📐 Similitud coseno

Dados dos vectores `u, v ∈ ℝ^d`, la similitud coseno es el coseno del ángulo entre ellos:

```
cos(u, v) = (u · v) / (‖u‖ · ‖v‖)      con  u · v = Σᵢ uᵢvᵢ  y  ‖u‖ = √(Σᵢ uᵢ²)
```

- Rango `[−1, 1]`; 1 = misma dirección, 0 = ortogonales, −1 = opuestos.
- Es **invariante a la escala**: `cos(u, 3v) = cos(u, v)`. Mide orientación, no magnitud.
- Si los vectores están **normalizados** (`‖u‖ = ‖v‖ = 1`), el coseno se reduce al producto punto, y ordenar por coseno equivale a ordenar por distancia euclídea, porque `‖u − v‖² = 2 − 2·cos(u, v)`. Por eso casi todos los índices normalizan.

🔍 Búsqueda exacta vs. aproximada (ANN)

La búsqueda **exacta** (índice *flat*) compara la consulta contra los `n` vectores: coste `O(n · d)` por consulta. Con `n = 10⁶` y `d = 768` son ~10^9 multiplicaciones: viable en batch, prohibitivo a baja latencia. Los índices **ANN** (*approximate nearest neighbor*) sacrifican exactitud garantizada por velocidad:

- **IVF** (*inverted file*): agrupa los vectores en `k` celdas por k-means; en consulta solo se exploran las `nprobe` celdas más cercanas al centroide de la consulta.
- **PQ** (*product quantization*): comprime cada vector en subvectores cuantizados, reduciendo memoria ~10-100× a costa de comparar contra aproximaciones.
- **HNSW** (*Hierarchical Navigable Small World*, Malkov & Yashunin, arXiv:1603.09320): un grafo de proximidad por capas. Las capas superiores tienen pocos nodos con aristas largas (autopistas); las inferiores, todos los nodos con aristas cortas. La búsqueda entra por arriba, avanza con voraz *greedy* hacia el vecino más cercano y desciende de capa, logrando complejidad empírica `O(log n)` con recall alto controlado por `efSearch`.

```
buscar_hnsw(q):
  nodo ← entrada de la capa superior
  para capa L..1:                    # descenso por autopistas
      nodo ← greedy_mas_cercano(q, nodo, capa)
  return busqueda_en_haz(q, nodo, capa 0, ef=efSearch)   # refinamiento final
```

La métrica clave de un índice ANN es **recall@k frente a latencia**: qué fracción de los verdaderos k vecinos devuelve y en cuánto tiempo. Nunca se reporta una sin la otra.

🧮 Ejemplo trabajado

Consulta y tres documentos ya embebidos en 3 dimensiones (didáctico; en la práctica `d` ≥ 256):

```
q  = (1, 2, 2)          ‖q‖  = √(1+4+4) = 3
d1 = (2, 4, 4)          ‖d1‖ = √(4+16+16) = 6
d2 = (0, 3, 4)          ‖d2‖ = √(0+9+16) = 5
d3 = (2, −1, 2)         ‖d3‖ = √(4+1+4) = 3

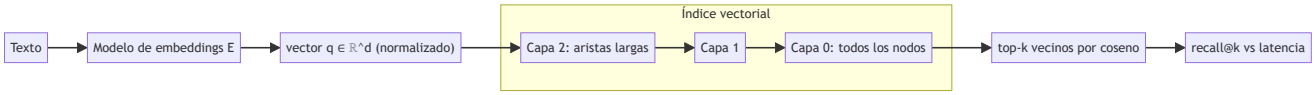
cos(q,d1) = (2+8+8)  / (3·6) = 18/18 = 1.000   ← misma dirección que q (d1 = 2q)
cos(q,d2) = (0+6+8)  / (3·5) = 14/15 ≈ 0.933
cos(q,d3) = (2−2+4)  / (3·3) = 4/9   ≈ 0.444

Ranking: d1 > d2 > d3
```

Obsérvese que `d1 = 2·q`: su magnitud es el doble pero el coseno es exactamente 1, mientras que la distancia euclídea `‖q − d1‖ = 3` no es cero. Coseno y euclídea solo coinciden en ranking cuando todos los vectores están normalizados.

📊 Propiedades y comparación

Índice	Búsqueda	Memoria	Recall	Inserciones	Uso típico
Flat (exacto)	`O(n·d)`	`O(n·d)`	100 %	triviales	n pequeño, baseline obligado
IVF	`O((n/k)·nprobe·d)`	`O(n·d)`	alto, depende de `nprobe`	requiere reentrenar centroides	colecciones medianas estáticas
IVF-PQ	sublineal	~10-100× menos	medio	ídem	miles de millones de vectores
HNSW	`O(log n)` empírico	`O(n·d + n·M)` (grafo)	muy alto (`efSearch`)	incrementales nativas	servicio online de baja latencia



⚠️ Errores conceptuales frecuentes

1. **"Coseno alto = relevancia"**. El coseno mide similitud semántica de superficie; una pregunta y su negación pueden tener coseno > 0.9. Relevancia exige señales adicionales (clase 104, re-ranking).
2. **Comparar embeddings de modelos distintos**. Cada modelo define su propio espacio; los vectores de dos modelos no son comparables ni promediables. Cambiar de modelo obliga a reindexar toda la colección.
3. **Tratar ANN como búsqueda exacta**. HNSW o IVF pueden omitir el verdadero vecino más cercano; si el recall@k no se mide contra un baseline flat, no se sabe cuánto se pierde.
4. **Ignorar la normalización**. Mezclar vectores normalizados y sin normalizar, o usar producto punto sobre vectores de normas muy dispares, produce rankings sesgados hacia documentos "largos" en norma.
5. **Intuición euclídea en alta dimensión**. Con `d` grande las distancias se concentran (todas parecen similares) y el volumen se acumula en la corteza: las intuiciones 2D/3D sobre "cercanía" fallan; por eso se valida con métricas, no con dibujos.

🚀 Del aprendizaje a la operación

Entre este núcleo y un buscador vectorial real faltan: la elección y evaluación del modelo de embeddings sobre el dominio propio (no el benchmark público), el pipeline de reindexado cuando el modelo cambia de versión, filtros de metadatos combinados con la búsqueda ANN (pre- vs post-filtrado), la operación del índice (memoria, réplicas, snapshots) y el monitoreo de recall en producción con consultas etiquetadas. Motores como faiss o Qdrant resuelven la infraestructura, no la calidad del embedding.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Malkov, Y. & Yashunin, D. (2016). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. arXiv:1603.09320 — uso: fuente primaria del mecanismo estudiado
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084 — uso: fuente primaria del mecanismo estudiado
- Johnson, J., Douze, M. & Jégou, H. (2017). *Billion-scale similarity search with GPUs* (faiss). arXiv:1702.08734 — uso: fuente primaria del mecanismo estudiado
- Jurafsky, D. & Martin, J. H. *Speech and Language Processing* (3.ª ed., borrador), cap. "Vector Semantics and Embeddings". https://web.stanford.edu/~jurafsky/slp3/ — uso: desarrollo extendido del tema
- Documentación oficial de faiss: https://faiss.ai/ — uso: referencia consultada en su fuente original
- Documentación oficial de Qdrant: https://qdrant.tech/documentation/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P05 · Estimación eficiente de representaciones de palabras en un espacio vectorial	2013	El significado distribucional se vuelve barato: vectores densos entrenables sobre miles de millones de palabras.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 08
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define embeddings y búsqueda vectorial sin usar una marca o framework como definición.
2. Explica la relación entre embeddings, vector search, cosine, índices.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

101 — Segmentación, metadatos y ventanas

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **segmentación, metadatos y ventanas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar segmentación, metadatos y ventanas usando los conceptos `chunking`, `metadata`, `overlap`, `estructura`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`chunking`, `metadata`, `overlap`, `estructura`

🗺️ Ubicación en el mapa de la IA

Un índice vectorial (clase 100) no busca documentos: busca fragmentos. Cómo se corta un documento en unidades indexables —el *chunking*— determina qué puede recuperarse y qué se pierde antes de que ninguna consulta exista. Esta clase aporta la capa de ingesta que todo sistema RAG (clases 105-111) presupone: segmentación, metadatos que permiten filtrar y atribuir, y ventanas que reconcilian "recuperar preciso" con "leer con contexto".

📖 Fundamentos

🎌 El problema del chunking

Los modelos de embeddings y los LLM tienen ventanas finitas, y un embedding de un documento entero diluye sus temas en un solo vector. Por eso se indexan **chunks**: fragmentos de tamaño acotado. El diseño enfrenta una tensión central:

- **Chunks pequeños** → embeddings más nítidos (un tema por vector), mejor precisión de recuperación, pero fragmentos ilegibles sin su entorno.

- **Chunks grandes** → más contexto autocontenido, pero embeddings "promedio" que recuperan peor y llenan la ventana del LLM con ruido.

🧩 Estrategias de segmentación

```
1. Tamaño fijo:        cortar cada N tokens (con solape S). Simple, ignora estructura.
2. Recursiva:          intentar cortar por separadores jerárquicos
                       (secciones → párrafos → oraciones → tokens) hasta caber en N.
3. Estructural:        respetar la estructura del formato (encabezados Markdown,
                       celdas de tabla, bloques de código) como fronteras duras.
4. Semántica:          cortar donde la similitud entre oraciones consecutivas cae
                       por debajo de un umbral (cambio de tema detectado con embeddings).
```

El **solape** (*overlap*) repite `S` tokens entre chunks consecutivos para que una idea que cruza la frontera exista completa en al menos un chunk. Coste: almacenamiento e indexación redundantes (factor `N/(N−S)`).

🪟 Ventanas: separar unidad de búsqueda y unidad de lectura

Dos patrones desacoplan el vector que se busca del texto que se entrega al LLM:

- **Sentence-window**: se indexa cada oración (búsqueda precisa) pero se devuelve la oración ± k vecinas (lectura con contexto).
- **Parent-document / chunk jerárquico**: se indexan chunks hijos pequeños; al recuperar un hijo se entrega su chunk padre (sección completa). La búsqueda apunta fino; la generación lee ancho.

🏷️ Metadatos

Cada chunk viaja con metadatos estructurados: `doc_id`, título, sección, posición, fecha, autor, idioma, permisos de acceso. Sirven para tres cosas que el vector no puede hacer: **filtrar** (solo documentos de 2024, solo los que este usuario puede ver), **atribuir** (citar fuente y sección exactas, clase 105) y **depurar** (rastrear por qué un chunk apareció). El filtrado puede ser *pre-filter* (el índice restringe el espacio antes de buscar, exacto pero exige soporte del motor) o *post-filter* (se recuperan más candidatos y se filtran después, simple pero puede quedarse corto de resultados).

🧮 Ejemplo trabajado

Documento de **900 tokens**, chunking de tamaño fijo con `N = 300` y solape `S = 50`. Cada chunk empieza donde el anterior avanzó `N − S = 250` tokens:

```
chunk 0: tokens [0, 300)      inicio = 0·250 = 0
chunk 1: tokens [250, 550)    inicio = 1·250 = 250   (repite 250-300 del chunk 0)
chunk 2: tokens [500, 800)    inicio = 2·250 = 500
chunk 3: tokens [750, 900)    inicio = 3·250 = 750   (parcial, 150 tokens)

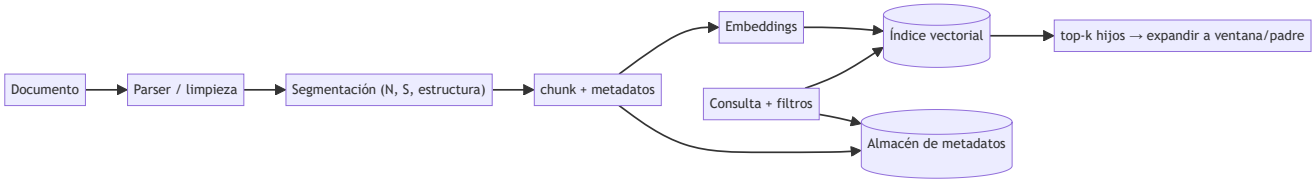
Número de chunks: ⌈(900 − 50) / 250⌉ = ⌈3.4⌉ = 4
Tokens almacenados: 300+300+300+150 = 1050 → sobrecoste 1050/900 ≈ 1.17×
```

Una afirmación que ocupe los tokens 240-310 quedaría cortada en un esquema sin solape (frontera en 300); con `S = 50` vive completa dentro del chunk 1 `[250, 550)` … salvo sus primeros 10 tokens. Moraleja: el solape

reduce el riesgo de corte, no lo elimina; solo las fronteras estructurales (párrafo, sección) lo evitan por construcción.

📊 Propiedades y comparación

Estrategia	Respeta estructura	Coste de ingesta	Riesgo de cortar ideas	Cuándo usarla
Tamaño fijo + solape	No	Mínimo	Medio (mitigado por solape)	baseline, texto homogéneo
Recursiva	Parcial	Bajo	Bajo-medio	documentos con párrafos claros
Estructural (Markdown/HTML)	Sí	Medio (parser)	Bajo	documentación técnica, wikis
Semántica	Implícita	Alto (embeddings en ingesta)	Bajo	texto largo sin estructura
Sentence-window / parent-doc	Sí (2 niveles)	Medio	Muy bajo en lectura	RAG con citas precisas



⚠️ Errores conceptuales frecuentes

1. **"Existe un tamaño de chunk óptimo universal"**. El óptimo depende del corpus, del modelo de embeddings y de la tarea; se elige midiendo recall sobre consultas propias (clase 110), no copiando un número de un tutorial.
2. **Chunking sin metadatos**. Un chunk sin `doc_id` ni posición no puede citarse ni deduplicarse; la atribución de la clase 105 se vuelve imposible de reconstruir a posteriori.
3. **Confundir unidad de búsqueda con unidad de lectura**. Optimizar un único tamaño para ambas cosas sacrifica una; sentence-window y parent-document existen precisamente para separarlas.
4. **Ignorar tablas y código**. Cortar una tabla por la mitad o un bloque de código por una línea arbitraria produce chunks sintácticamente inválidos que el embedding representa mal y el LLM malinterpreta.
5. **Solape como solución mágica**. El solape duplica contenido (sesga el top-k hacia documentos con más chunks casi idénticos) y no protege las ideas más largas que `S`.

🚀 Del aprendizaje a la operación

En producción el chunking es un pipeline versionado: cambiar `N`, `S` o la estrategia obliga a reindexar y a invalidar caches, así que cada chunk guarda la versión de su receta de segmentación. Faltan además: deduplicación entre documentos casi idénticos, manejo de PDFs con layout complejo (tablas, columnas),

propagación de permisos del documento a cada chunk (seguridad a nivel de fila) y métricas de ingesta (chunks vacíos, demasiado cortos, sin metadatos) antes de que ninguna consulta los delate.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (los documentos se indexan como pasajes de ~100 palabras). arXiv:2005.11401 — uso: fuente primaria del mecanismo estudiado
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv:2004.04906 — uso: fuente primaria del mecanismo estudiado
- Jurafsky, D. & Martin, J. H. *Speech and Language Processing* (3.ª ed., borrador), cap. sobre recuperación y RAG. https://web.stanford.edu/~jurafsky/slp3/ — uso: desarrollo extendido del tema
- Documentación oficial de Qdrant — payloads y filtrado: https://qdrant.tech/documentation/concepts/payload/ — uso: referencia consultada en su fuente original
- Documentación oficial de LangChain — text splitters: https://python.langchain.com/docs/concepts/text_splitters/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P36 · Perdidos en el medio: cómo usan los modelos de lenguaje los contextos largos	2023	Tener contexto largo no es usarlo: el rendimiento cae en forma de U cuando el dato relevante está en el medio.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 08
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define segmentación, metadatos y ventanas sin usar una marca o framework como definición.
2. Explica la relación entre chunking, metadata, overlap, estructura.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

102 — Búsqueda léxica y BM25

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **búsqueda léxica y bm25** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar búsqueda léxica y bm25 usando los conceptos `BM25`, `inverted index`, `sparse`, `tokens`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`BM25`, `inverted index`, `sparse`, `tokens`

🗺️ Ubicación en el mapa de la IA

Antes de los embeddings, la recuperación de información se construyó sobre estadísticas de términos: TF-IDF en los años 70 y su refinamiento probabilístico BM25 en los 90 (Robertson et al., proyecto Okapi). Lejos de quedar obsoleta, la búsqueda léxica sigue siendo el complemento exacto de la vectorial (clase 100): encuentra identificadores, nombres propios y términos raros que los embeddings difuminan. La clase 103 fusionará ambas; esta clase da la mitad léxica con su fórmula y sus porqués.

📖 Fundamentos

📚 Índice invertido y modelo de bolsa de palabras

La búsqueda léxica trata documento y consulta como **bolsas de términos** (sin orden) y se apoya en un **índice invertido**: para cada término, la lista de documentos donde aparece con su frecuencia. Buscar no recorre documentos: interseca listas de términos, por eso escala a colecciones enormes.

🧮 La fórmula BM25

Para una consulta $Q = \{t_1 \dots t_m\}$ y un documento D (Robertson & Zaragoza, 2009):

```
score(D, Q) = Σ_{t∈Q}  IDF(t) · ( f(t,D) · (k₁ + 1) ) / ( f(t,D) + k₁ · (1 − b + b · |D|/avgdl) )

IDF(t) = ln( 1 + (N − df(t) + 0.5) / (df(t) + 0.5) )
```

donde `f(t,D)` es la frecuencia del término en el documento, `|D|` la longitud del documento en tokens, `avgdl` la longitud media de la colección, `N` el número de documentos y `df(t)` en cuántos aparece el término. Cada pieza tiene un motivo:

- **IDF**: los términos raros discriminan más. Un término presente en casi todos los documentos (`df ≈ N`) aporta casi cero; uno raro aporta mucho.
- **Saturación de TF** (parámetro `k₁ ≈ 1.2-2.0`): la ganancia de repetir un término crece cóncavamente y se aplana en `k₁ + 1`. La décima aparición de "gato" vale mucho menos que la primera: repetir palabras no debe dominar el ranking.
- **Normalización por longitud** (parámetro `b ∈ [0,1]`, típico 0.75): un documento largo acumula términos por puro volumen; el factor `1 − b + b·|D|/avgdl` penaliza a los más largos que la media y premia a los más cortos. Con `b = 0` no se normaliza; con `b = 1`, normalización completa.

BM25 proviene del **marco probabilístico de relevancia** (Probability Ranking Principle): ordenar por probabilidad estimada de relevancia. La fórmula es una aproximación práctica de ese principio con supuestos de independencia entre términos.

⚖️ Fortalezas y límites frente a lo denso

BM25 es exacto con lo literal: códigos de error, números de pieza, apellidos, siglas. No requiere entrenamiento, es interpretable (se puede descomponer el score término a término) y barato. Su límite es el **desajuste de vocabulario** (*vocabulary mismatch*): "coche" no recupera "automóvil". Justo donde los embeddings brillan; de ahí la hibridación de la clase 103.

🧮 Ejemplo trabajado

Colección de `N = 3` documentos, consulta `Q = {gato, negro}`, con `k₁ = 1.2`, `b = 0.75`:

```
D1: "el gato negro duerme"                    |D1| = 4   f(gato)=1  f(negro)=1
D2: "el gato blanco y el gato gris juegan"    |D2| = 8   f(gato)=2  f(negro)=0
D3: "el perro negro corre y ladra"            |D3| = 6   f(gato)=0  f(negro)=1

avgdl = (4+8+6)/3 = 6
df(gato) = 2, df(negro) = 2  →  IDF = ln(1 + (3−2+0.5)/(2+0.5)) = ln(1.6) ≈ 0.470

D1: factor de longitud = 1.2·(1−0.75 + 0.75·4/6) = 1.2·0.75 = 0.9
    gato:  0.470 · (1·2.2)/(1+0.9)  = 0.470·1.158 ≈ 0.544
    negro: idéntico                              ≈ 0.544     score(D1) ≈ 1.088

D2: factor = 1.2·(0.25 + 0.75·8/6) = 1.5
    gato:  0.470 · (2·2.2)/(2+1.5)  = 0.470·1.257 ≈ 0.591    score(D2) ≈ 0.591

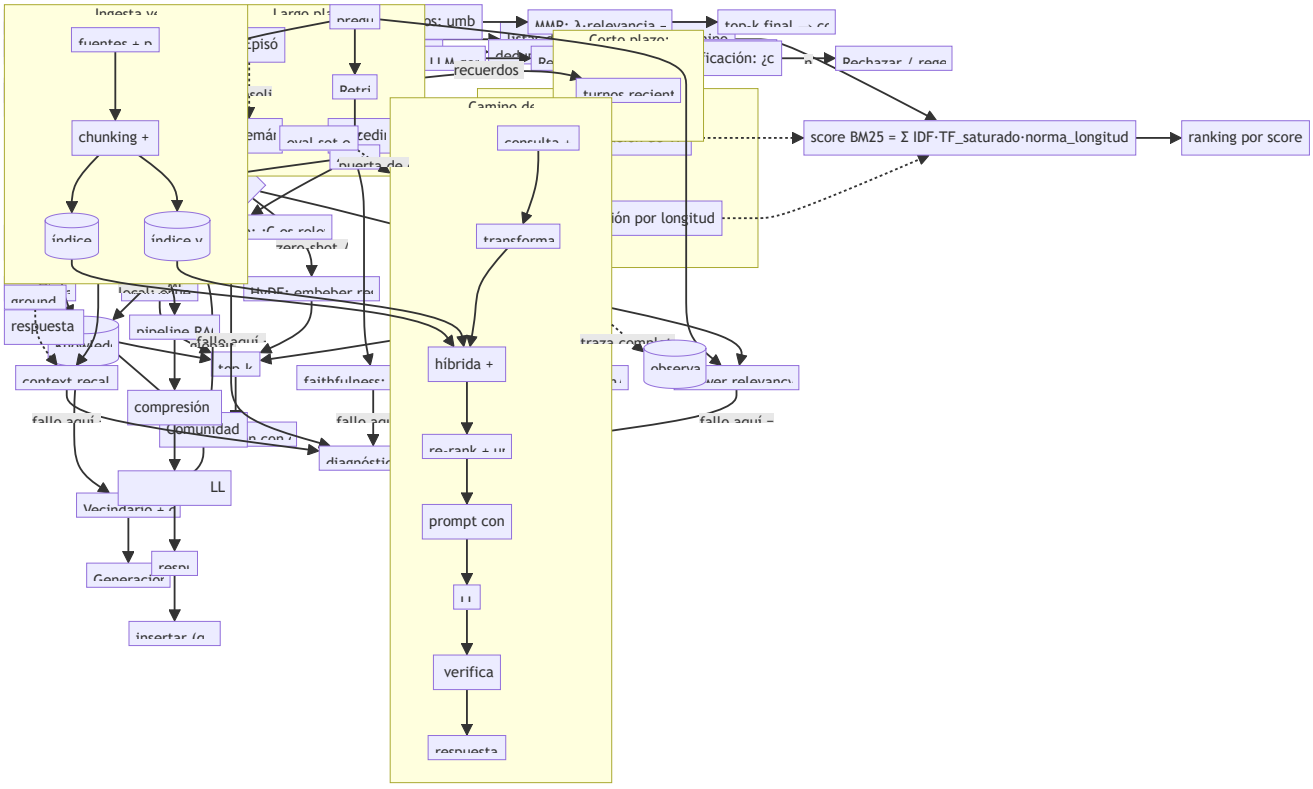
D3: factor = 1.2·(0.25 + 0.75·6/6) = 1.2
    negro: 0.470 · (1·2.2)/(1+1.2)  = 0.470·1.000 = 0.470    score(D3) ≈ 0.470

Ranking: D1 (1.088) > D2 (0.591) > D3 (0.470)
```

Lección del ejemplo: D2 contiene "gato" **dos veces** y aun así pierde contra D1, que cubre **ambos** términos una vez. La saturación de TF hace que cubrir más términos de la consulta valga más que repetir uno; además D2 paga su longitud (8 > avgdl).

📊 Propiedades y comparación

Propiedad	BM25 (léxico)	Embeddings (denso)
Coincidencia	literal, término exacto	semántica, sinónimos y paráfrasis
Términos raros / IDs / siglas	excelente	débil (se difuminan en el vector)
Vocabulary mismatch	falla	resuelve
Entrenamiento	ninguno	modelo preentrenado (y su versionado)
Interpretabilidad	score descomponible por término	opaco (vector denso)
Coste de índice	invertido, compacto	vectorial (memoria + ANN)
Multilingüe / dominio nuevo	funciona de inmediato	depende del modelo



⚠️ Errores conceptuales frecuentes

1. **"BM25 está obsoleto; lo denso siempre gana"**. En dominios con jerga, códigos o consultas cortas y literales, BM25 sigue siendo un baseline durísimo de batir; los sistemas serios lo combinan, no lo descartan.
2. **Confundir BM25 con TF-IDF**. TF-IDF crece linealmente con la frecuencia; BM25 satura la TF y normaliza por longitud con parámetros ajustables. No son la misma fórmula.

3. **Ignorar k_1 y b**. Los valores por defecto (1.2, 0.75) provienen de colecciones TREC; en corpus con longitudes atípicas (tuits, contratos) ajustarlos cambia el ranking.
4. **Olvidar la tokenización**. BM25 opera sobre términos: sin minúsculas, *stemming* o manejo de acentos coherentes entre índice y consulta, "Gato" y "gato" son términos distintos.
5. **Leer el score como probabilidad**. El score BM25 no está acotado ni calibrado; solo ordena dentro de una misma consulta. Comparar scores entre consultas distintas es incorrecto (esto motiva RRF en la clase 103).

🚀 Del aprendizaje a la operación

Un BM25 de producción vive en un motor (Lucene/Elasticsearch/OpenSearch) con analizadores por idioma, sinónimos gestionados, *boosting* por campos (título > cuerpo) y ajuste de k_1 / b validado con consultas etiquetadas. Faltan además la actualización incremental del índice, la coherencia de tokenización entre ingesta y consulta, y la decisión de cuándo BM25 actúa solo, cuándo como candidato para re-ranking (clase 104) y cuándo fusionado con lo denso (clase 103).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4). DOI 10.1561/1500000019 — uso: fuente primaria del mecanismo estudiado
- Robertson, S. et al. (1994). *Okapi at TREC-3* — origen del esquema BM25 en las campañas TREC: https://trec.nist.gov/pubs/trec3/t3_proceedings.html — uso: referencia consultada en su fuente original
- Jurafsky, D. & Martin, J. H. *Speech and Language Processing* (3.ª ed., borrador), cap. de recuperación de información. https://web.stanford.edu/~jurafsky/slp3/ — uso: desarrollo extendido del tema
- Manning, C., Raghavan, P. & Schütze, H. *Introduction to Information Retrieval* (libro gratuito oficial): https://nlp.stanford.edu/IR-book/ — uso: referencia consultada en su fuente original

- Documentación oficial de Apache Lucene (implementación de referencia de BM25): https://lucene.apache.org/core/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 08
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 08

📝 Evaluación completa

❓ Preguntas

1. Define búsqueda léxica y bm25 sin usar una marca o framework como definición.
2. Explica la relación entre BM25, inverted index, sparse, tokens.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.

5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

103 — Búsqueda híbrida y fusión de rankings

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **búsqueda híbrida y fusión de rankings** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar búsqueda híbrida y fusión de rankings usando los conceptos `hybrid`, `RRF`, `sparse`, `dense`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`hybrid`, `RRF`, `sparse`, `dense`

🗺️ Ubicación en el mapa de la IA

Las clases 097 y 099 dejaron dos recuperadores con fallos complementarios: el denso resuelve sinónimos pero difumina identificadores; BM25 clava lo literal pero sufre el desajuste de vocabulario. La búsqueda híbrida los ejecuta en paralelo y **fusiona sus rankings**, y es hoy el punto de partida estándar de los sistemas RAG serios. El obstáculo técnico —scores incomparables entre motores— se resuelve con fusión por posiciones (RRF), la técnica central de esta clase; la 101 refinará el resultado con re-ranking.

📖 Fundamentos

🔀 Por qué fusionar rankings y no scores

Un score BM25 no está acotado y depende de la colección; un coseno vive en `[-1, 1]`. Sumarlos directamente es sumar peras con manzanas: el motor con la escala más grande domina. Dos familias de solución:

- **Normalizar y combinar** (*score fusion*): llevar cada score a `[0,1]` (min-max sobre el top-k) y combinar `s = α·s_denso + (1−α)·s_léxico`. Funciona, pero `α` y la normalización dependen de cada consulta y colección: frágil.

- **Fusión por posiciones** (*rank fusion*): descartar los scores y usar solo el **puesto** de cada documento en cada ranking. Robusta, sin calibración.

🧮 Reciprocal Rank Fusion (RRF)

Propuesta por Cormack, Clarke y Büttcher (SIGIR 2009):

```
RRF(d) = Σ_r∈rankings  1 / (k + rank_r(d))
```

donde `rank_r(d)` es la posición (1 = primero) del documento `d` en el ranking `r`, y `k` es una constante de suavizado (típicamente **60**). Un documento ausente de un ranking simplemente no suma ese término. Los porqués:

- El **recíproco** concentra el crédito en las primeras posiciones: pasar del puesto 1 al 2 pesa más que del 50 al 51.
- La constante `k` amortigua esa cima: sin ella (`k = 0`), estar 1.º en un solo ranking (1/1 = 1.0) aplastaría cualquier consenso. Con `k = 60`, el puesto 1 vale 1/61 y el 10 vale 1/70: la diferencia existe pero no dicta sola el resultado. `k` grande → fusión más "democrática" entre posiciones.
- Al usar solo posiciones, RRF es inmune a escalas, outliers y calibraciones de score: puede fusionar BM25, denso, e incluso motores de terceros de los que solo se conoce el orden.

```
fusion_rrf(rankings, k=60):
    puntaje = defaultdict(0)
    para r en rankings:
        para (puesto, doc) en enumerate(r, desde=1):
            puntaje[doc] += 1 / (k + puesto)
    return documentos ordenados por puntaje desc
```

🎛️ Diseño de un recuperador híbrido

Decisiones habituales: cuántos candidatos pide cada rama (p. ej. top-50 de cada una), si alguna rama lleva peso extra (RRF ponderado: `w_r/(k + rank)`), cómo se deduplican chunks del mismo documento, y qué presupuesto final pasa al re-ranker o al LLM. La evaluación (clase 110) compara siempre híbrido contra cada rama sola: la fusión debe ganar o no se justifica su coste doble.

🧮 Ejemplo trabajado

Dos rankings sobre la misma consulta, `k = 60`:

```
BM25:      [A, B, C, D]         Denso:     [C, A, E, B]

RRF(A) = 1/(60+1) + 1/(60+2) = 0.01639 + 0.01613 = 0.03252
RRF(B) = 1/(60+2) + 1/(60+4) = 0.01613 + 0.01563 = 0.03175
RRF(C) = 1/(60+3) + 1/(60+1) = 0.01587 + 0.01639 = 0.03227
RRF(D) = 1/(60+4)            = 0.01563              (solo aparece en BM25)
RRF(E) = 1/(60+3)            = 0.01587              (solo aparece en denso)

Fusión: A (0.03252) > C (0.03227) > B (0.03175) > E (0.01587) > D (0.01563)
```

Lectura: **A** gana sin ser 1.º en ningún ranking, porque está en el top-2 de ambos: RRF premia el consenso. **C**, primero en el denso pero tercero en BM25, queda segundo. Los documentos vistos por un solo motor (D, E) caen al fondo — aparecer en ambas ramas es una señal fuerte de relevancia.

📊 Propiedades y comparación

Método de fusión	Usa scores	Necesita calibración	Robustez entre motores	Ajustables
Suma de scores crudos	Sí	—	muy baja (escalas dispares)	ninguno
Min-max + combinación convexa	Sí	por consulta/colección	media	α, normalización
RRF	No (solo posiciones)	no	alta	k (y pesos w_r opcionales)
Re-ranking neuronal (clase 104)	recalcula	modelo entrenado	alta	modelo, presupuesto

⚠️ Errores conceptuales frecuentes

1. **Sumar scores BM25 y cosenos directamente**. Escalas incomparables: el resultado lo decide la aritmética de las magnitudes, no la relevancia. Normaliza o usa posiciones.
2. **"RRF necesita los scores"**. Al contrario: solo necesita el orden. Esa es su ventaja operativa — funciona con cualquier motor que devuelva una lista ordenada.
3. **Tratar k = 60 como ley**. Es un valor empírico razonable del paper original; con rankings muy cortos o muchas ramas, conviene validarlo con consultas etiquetadas.
4. **Fusionar sin deduplicar**. Si ambos motores devuelven chunks distintos del mismo documento, la fusión puede llenar el top-k con un solo documento repetido.
5. **Asumir que híbrido siempre gana**. Si una rama es mala (índice desactualizado, embeddings fuera de dominio), la fusión puede degradar a la rama buena; la comparación contra cada rama sola es parte del contrato de evaluación.

🚀 Del aprendizaje a la operación

En producción la hibridación añade: ejecución en paralelo con presupuestos de latencia por rama (y qué hacer si una rama expira), pesos por tipo de consulta (una consulta con comillas o códigos puede inclinar hacia

BM25), RRF ponderado ajustado con datos de clics o juicios, y monitoreo separado de cada rama para detectar cuál se degrada. Motores como OpenSearch, Qdrant o Vespa ya traen fusión híbrida nativa; la decisión de diseño sigue siendo qué ramas, cuántos candidatos y cómo se evalúa la ganancia.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Cormack, G., Clarke, C. & Büttcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. SIGIR 2009. DOI 10.1145/1571941.1572114 — uso: fuente primaria del mecanismo estudiado
- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. DOI 10.1561/1500000019 — uso: fuente primaria del mecanismo estudiado
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv:2004.04906 — uso: fuente primaria del mecanismo estudiado
- Manning, C., Raghavan, P. & Schütze, H. *Introduction to Information Retrieval*: https://nlp.stanford.edu/IR-book/ — uso: referencia consultada en su fuente original
- Documentación oficial de Qdrant — búsqueda híbrida: https://qdrant.tech/documentation/concepts/hybrid-queries/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 08
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define búsqueda híbrida y fusión de rankings sin usar una marca o framework como definición.
2. Explica la relación entre hybrid, RRF, sparse, dense.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

104 — Re-ranking y filtros de evidencia

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **re-ranking y filtros de evidencia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar re-ranking y filtros de evidencia usando los conceptos `reranking`, `cross-encoder`, `filtros`, `calidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`reranking`, `cross-encoder`, `filtros`, `calidad`

🗺️ Ubicación en el mapa de la IA

La recuperación de las clases 100-103 optimiza el *recall*: traer candidatos plausibles rápido y barato. El re-ranking optimiza la *precisión*: reordenar esos candidatos con un modelo más caro y preciso que ya puede leer consulta y documento juntos. Este patrón de dos etapas —recuperar amplio, refinar caro— viene de los buscadores web clásicos y es hoy la arquitectura estándar de todo pipeline RAG serio: la calidad del contexto que llega al generador (clase 105) depende directamente de este filtro.

📖 Fundamentos

⚖️ Bi-encoder vs. cross-encoder

Un **bi-encoder** (como los modelos de embeddings de la clase 100) codifica consulta y documento **por separado** en vectores, y la relevancia se estima con una función barata (coseno). Eso permite pre-computar los vectores de toda la colección e indexarlos: coste por consulta `O(1)` embeddings + búsqueda ANN. El precio es expresividad: el modelo nunca ve consulta y documento juntos, así que no puede modelar interacciones finas (negaciones, coincidencia de entidades, condiciones).

Un **cross-encoder** (Nogueira & Cho, arXiv:1901.04085) concatena consulta y documento en una sola entrada — `[CLS] consulta [SEP] documento` — y un transformer produce un **score de relevancia** con atención completa entre todos los tokens de ambos textos. Es mucho más preciso, pero no hay nada pre-computable: cada par (q, d) exige una pasada completa del modelo. Con `n = 10⁶` documentos es inviable como primera etapa; con los `k = 50` candidatos de la primera etapa, es trivial.

```
Pipeline retrieve-then-rerank:
  1. Recuperación (bi-encoder / BM25 / híbrida)  → top-100 candidatos     (barato, alto recall)
  2. Re-ranking (cross-encoder sobre 100 pares)  → top-10 reordenados     (caro, alta precisión)
  3. Filtros de evidencia                        → top-k final al LLM     (umbral, dedup, diversidad)
```

🔀 MMR: relevancia con diversidad

Los k mejores documentos por score suelen ser redundantes (variantes del mismo pasaje). **Maximal Marginal Relevance** (Carbonell & Goldstein, 1998, DOI 10.1145/290941.291025) selecciona iterativamente el documento que maximiza un compromiso entre relevancia y novedad:

```
MMR(d) = λ · sim(q, d) − (1 − λ) · max_{s ∈ S} sim(d, s)
```

donde `S` es el conjunto ya seleccionado y `λ ∈ [0, 1]` controla el equilibrio: `λ = 1` es ranking puro por relevancia; `λ = 0` maximiza solo diversidad. En cada iteración se recalcula el término de penalización porque `S` creció.

🧹 Filtros de evidencia

Después del re-ranking, antes de pasar contexto al generador, se aplican filtros que convierten "candidatos ordenados" en "evidencia admisible":

- **Umbral de score**: descartar documentos bajo un mínimo calibrado; con score bajo es mejor entregar k' < k documentos (o ninguno y declarar "no hay evidencia") que rellenar.
- **Deduplicación**: eliminar pasajes casi idénticos (similitud entre documentos > 0.95).
- **Filtros de metadatos**: fecha, fuente autorizada, permisos de acceso del usuario.
- **Presupuesto de tokens**: el contexto del LLM es finito; k se elige por presupuesto, no por costumbre.

El principio: es preferible un contexto corto y limpio que uno largo y ruidoso, porque el generador no distingue por sí solo la evidencia buena de la mala.

🧮 Ejemplo trabajado

MMR a mano con `λ = 0.7`, k = 2, y cuatro candidatos ya puntuados:

```
sim(q,·):   d1 = 0.90   d2 = 0.85   d3 = 0.70   d4 = 0.60
sim entre documentos:  sim(d1,d2) = 0.95 (casi duplicados)
                       sim(d1,d3) = 0.30   sim(d1,d4) = 0.20
                       sim(d2,d3) = 0.35   sim(d2,d4) = 0.25

Iteración 1 (S = ∅, sin penalización): gana el más relevante → d1.  S = {d1}

Iteración 2 (penaliza similitud con d1):
  MMR(d2) = 0.7·0.85 − 0.3·0.95 = 0.595 − 0.285 = 0.310
  MMR(d3) = 0.7·0.70 − 0.3·0.30 = 0.490 − 0.090 = 0.400   ← gana
  MMR(d4) = 0.7·0.60 − 0.3·0.20 = 0.420 − 0.060 = 0.360

Selección final: {d1, d3}
```

Nótese que d2, el segundo más relevante, **queda fuera**: su casi-duplicidad con d1 (0.95) lo penaliza más de lo que su relevancia lo favorece. Con `λ = 0.9` la penalización pesaría 0.1 y d2 ganaría (0.765 − 0.095 = 0.670 > 0.603 de d3): λ decide qué significa "mejor contexto".

📊 Propiedades y comparación

Arquitectura	Interacción q-d	Pre-cómputo	Coste por consulta	Calidad	Rol en el pipeline
Bi-encoder	ninguna (vectores separados)	sí (índice)	muy bajo	media	1.ª etapa, recall
Late interaction (ColBERT)	por token, tardía	parcial	medio	media-alta	1.ª etapa mejorada
Cross-encoder	atención completa	no	alto (1 pasada por par)	alta	2.ª etapa, precisión
MMR	entre documentos	no	bajo ($O(k^2)$ sims)	diversidad	selección final

⚠️ Errores conceptuales frecuentes

1. **"El cross-encoder es mejor, úsalo para todo"**. Sin primera etapa es inviable: puntuar 10^6 pares por consulta son horas de GPU. Su rol es refinar decenas, no explorar millones.
2. **Re-rankear no mejora el recall**. Si el documento correcto no está en el top-100 de la primera etapa, ningún re-ranker lo recuperará: el re-ranking solo reordena. Un recall@100 bajo se arregla en la etapa 1, no en la 2.

3. **Comparar scores de cross-encoder entre consultas**. El score es un ordinal dentro de la misma consulta; 0.62 en una consulta y 0.62 en otra no significan la misma relevancia. Los umbrales requieren calibración sobre datos propios.
4. **MMR con λ arbitrario**. λ = 0.5 "por defecto" puede excluir el documento más útil. λ se ajusta observando el efecto en la calidad de la respuesta final, no se hereda.
5. **Filtrar después de truncar**. Aplicar el presupuesto de tokens antes de deduplicar desperdicia contexto en repeticiones; el orden correcto es: score → dedup → diversidad → presupuesto.

🚀 Del aprendizaje a la operación

Entre este núcleo y un re-ranker en producción faltan: elegir y evaluar un modelo de cross-encoder sobre pares etiquetados del dominio propio (no MS MARCO), medir la latencia añadida por la segunda etapa bajo carga real (suele dominar el p95 del pipeline), calibrar umbrales de score con validación periódica, decidir el comportamiento cuando ningún candidato supera el umbral (responder "sin evidencia" es una función del producto), y monitorear la deriva: un corpus que crece cambia la distribución de scores.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv:1901.04085 — uso: fuente primaria del mecanismo estudiado
- Carbonell, J. & Goldstein, J. (1998). *The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries*. SIGIR '98. DOI 10.1145/290941.291025 — uso: fuente primaria del mecanismo estudiado
- Khattab, O. & Zaharia, M. (2020). *ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT*. arXiv:2004.12832 — uso: fuente primaria del mecanismo estudiado
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv:1908.10084 — uso: fuente primaria del mecanismo estudiado

- Documentación de cross-encoders en sentence-transformers: https://www.sbert.net/examples/applications/cross-encoder/README.html — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define re-ranking y filtros de evidencia sin usar una marca o framework como definición.
2. Explica la relación entre reranking, cross-encoder, filtros, calidad.

3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

105 — RAG básico con citas

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **rag básico con citas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rag básico con citas usando los conceptos `RAG`, `citas`, `grounding`, `corpus`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`RAG`, `citas`, `grounding`, `corpus`

🗺️ Ubicación en el mapa de la IA

RAG (*Retrieval-Augmented Generation*, Lewis et al., 2020) une las dos ramas que este programa venía desarrollando por separado: la recuperación de información (clases 097-101) y la generación con LLMs (parte 06). Nació para dar a los modelos acceso a conocimiento actualizable sin reentrenar, y se convirtió en el patrón dominante para reducir alucinaciones y dar trazabilidad a las respuestas. Todo lo que sigue en esta parte —consultas transformadas, grafos, memoria, evaluación— son refinamientos de este esqueleto.

📖 Fundamentos

🔗 Qué es RAG

RAG condiciona la generación de un LLM sobre documentos recuperados en tiempo de consulta. El paper original (arXiv:2005.11401) formuló la probabilidad de la respuesta `y` dada la consulta `x` marginalizando sobre documentos `z` recuperados por un retriever denso (DPR):

```
p(y | x) = Σ_z  p_retriever(z | x) · p_generator(y | x, z)
```

- **RAG-Sequence**: un mismo documento condiciona toda la secuencia generada.
- **RAG-Token**: cada token puede apoyarse en un documento distinto.

El RAG moderno con LLMs congela ambos componentes y opera por *prompting*: recuperar top-k pasajes, insertarlos en el prompt con identificadores, y pedir la respuesta. La distinción clave frente al conocimiento **paramétrico** (lo memorizado en los pesos): el conocimiento recuperado es **actualizable** (reindexar, no reentrenar), **inspeccionable** (se sabe qué se le mostró al modelo) y **atribuible** (se puede citar).

📎 Groundedness y citas

Una respuesta está **fundamentada** (*grounded*) si cada afirmación que contiene se sustenta en el contexto recuperado. El mecanismo estándar:

1. Numerar los pasajes en el prompt: `[1] texto…`, `[2] texto…`.
2. Instruir: "responde solo con la información de los pasajes y cita cada afirmación con su identificador `[n]`; si la información no está, dilo".
3. Verificar después (clase 110): descomponer la respuesta en afirmaciones y comprobar que cada `[n]` citado realmente **implica** la afirmación que acompaña.

Una cita es una **afirmación verificable de procedencia**, no una decoración: citar un pasaje que no sostiene la frase es peor que no citar, porque fabrica confianza.

🧱 Anatomía del prompt RAG

```
[Sistema]  Eres un asistente que responde SOLO con el contexto dado.
[Contexto] [1] "La ley entró en vigor en marzo de 2021…"
           [2] "El plazo de apelación es de 30 días hábiles…"
[Pregunta] ¿Cuándo entró en vigor la ley y cuál es el plazo de apelación?
[Regla]    Cita cada dato como [n]. Si falta información, responde "no consta en el contexto".
```

Decisiones de diseño que cambian el resultado: cuántos pasajes (k), en qué orden (los LLMs atienden mejor a los extremos del contexto — "lost in the middle"), qué hacer si la recuperación viene vacía (rechazar es una respuesta válida), y cómo separar la instrucción del contenido recuperado para resistir instrucciones inyectadas en los documentos.

🧮 Ejemplo trabajado

Corpus mínimo de 3 pasajes y una pregunta:

```
[1] "El telescopio espacial James Webb se lanzó el 25 de diciembre de 2021."
[2] "El Webb observa principalmente en el infrarrojo, a diferencia del Hubble."
[3] "El Hubble se lanzó en 1990 a bordo del transbordador Discovery."

Pregunta: ¿Cuándo se lanzó el James Webb y en qué rango observa?

Respuesta generada:
"El James Webb se lanzó el 25 de diciembre de 2021 [1] y observa
principalmente en el infrarrojo [2]."

Verificación afirmación por afirmación:
  A1 "se lanzó el 25-12-2021"  → ¿[1] la implica?  SÍ
  A2 "observa en el infrarrojo" → ¿[2] la implica?  SÍ
  → groundedness = 2/2 = 1.0
```

Contraejemplo: si la respuesta añadiera "...y costó 10 000 millones de dólares [3]", la afirmación A3 no está en ningún pasaje (y [3] habla del Hubble): es una **alucinación con cita falsa**, groundedness = 2/3 ≈ 0.67. El dato puede ser cierto en el mundo; sigue siendo infundado respecto al corpus, que es lo que RAG promete.

📊 Propiedades y comparación

Estrategia	Conocimiento	Actualización	Atribución	Coste dominante	Riesgo principal
Solo paramétrico	en los pesos	reentrenar/fine-tune	imposible	entrenamiento	alucinación inverificable
RAG	corpus indexado	reindexar	por pasaje citado	recuperación + contexto	contexto irrelevante o ruidoso
Fine-tuning + RAG	ambos	mixta	parcial	ambos	atribuir al corpus lo que vino de los pesos
Contexto largo (sin índice)	documentos en el prompt	por consulta	difusa	tokens por consulta	coste y "lost in the middle"

⚠️ Errores conceptuales frecuentes

1. **"RAG elimina las alucinaciones"**. Las reduce y las vuelve auditables. El modelo puede ignorar el contexto, mezclarlo con memoria paramétrica o citar mal; sin verificación posterior no hay garantía.
2. **"La cita prueba la afirmación"**. La cita es una hipótesis de procedencia que hay que verificar: los modelos generan identificadores `[n]` sintácticamente correctos y semánticamente falsos.

3. **"Más pasajes = mejor respuesta"**. Pasado cierto k, el contexto extra diluye la señal y sube el coste; la posición media del contexto es la peor atendida (arXiv:2307.03172).
4. **Confundir "correcto" con "fundamentado"**. Una respuesta verdadera pero ausente del corpus es un fallo de RAG: la promesa del sistema es "esto sale de estas fuentes", no "esto es verdad".
5. **Tratar el corpus como confiable por defecto**. Los documentos recuperados pueden contener errores o instrucciones maliciosas (inyección indirecta); el generador los leerá con la misma autoridad que el resto del prompt si no se aísla.

🚀 Del aprendizaje a la operación

Entre este núcleo y un RAG operativo faltan: la ingesta continua con detección de documentos modificados y reindexado incremental, control de acceso por documento (el usuario no debe recibir citas de fuentes que no puede leer), defensa contra inyección indirecta en el corpus, política explícita de rechazo cuando la evidencia es insuficiente, evaluación continua de fidelidad y cobertura (clase 110) y trazas que permitan auditar meses después qué pasajes exactos produjeron cada respuesta (clase 111).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401 — uso: fuente primaria del mecanismo estudiado
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. arXiv:2004.04906 — uso: fuente primaria del mecanismo estudiado
- Izacard, G. & Grave, E. (2020). *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering* (Fusion-in-Decoder). arXiv:2007.01282 — uso: fuente primaria del mecanismo estudiado
- Liu, N. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172 — uso: fuente primaria del mecanismo estudiado

- Gao, Y. et al. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997 — uso: fuente primaria del mecanismo estudiado
- Gao, T. et al. (2023). *Enabling Large Language Models to Generate Text with Citations* (ALCE). arXiv:2305.14627 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define rag básico con citas sin usar una marca o framework como definición.

2. Explica la relación entre RAG, citas, grounding, corpus.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

106 — Transformación y descomposición de consultas

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **transformación y descomposición de consultas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar transformación y descomposición de consultas usando los conceptos `query rewrite`, `decomposition`, `routing`, `retrieval`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`query rewrite`, `decomposition`, `routing`, `retrieval`

🗺️ Ubicación en el mapa de la IA

Las clases 100-105 asumen que la consulta del usuario es una buena consulta de recuperación. Casi nunca lo es: es ambigua, coloquial, multi-hop o depende del historial de la conversación. Esta clase inserta una capa de **transformación de consultas** entre el usuario y el retriever — la misma idea que la expansión de consultas de la IR clásica, pero ejecutada ahora por un LLM. Es el precedente directo del *agentic RAG*: cuando la transformación se vuelve iterativa y decide qué buscar a continuación, el pipeline se convierte en agente (parte 09).

📖 Fundamentos

✍️ Reescritura (query rewriting)

El primer problema es el desajuste entre cómo pregunta el usuario y cómo está escrito el corpus. Un LLM reescribe la consulta antes de recuperar: resolver correferencias con el historial ("¿y en 2023?" → "ingresos de la empresa X en 2023"), eliminar cortesías y ruido, expandir siglas. El esquema *rewrite-retrieve-read*

(arXiv:2305.14283) demostró que entrenar/ajustar el reescritor mejora el resultado final más que tocar el retriever.

🔀 Multi-query y fusión

Una sola formulación puede caer en una región pobre del espacio de embeddings. La técnica **multi-query** genera `m` paráfrasis de la consulta, recupera con cada una y **fusiona** los rankings (típicamente con RRF, clase 103):

```
q → LLM → {q1, q2, q3}          # paráfrasis con vocabulario distinto
top-k(q1) ∪ top-k(q2) ∪ top-k(q3) → RRF → top-k final
```

Aumenta el recall a cambio de `m` recuperaciones y el coste de generar las variantes.

📄 HyDE: documentos hipotéticos

HyDE (*Hypothetical Document Embeddings*, arXiv:2212.10496) ataca la asimetría pregunta-documento: una pregunta corta y un pasaje de respuesta viven en regiones distintas del espacio vectorial. En lugar de embeber la pregunta, se pide al LLM que **escriba una respuesta hipotética** (que puede contener errores factuales) y se embebe ese texto: lo que importa no es su veracidad sino su **forma** — se parece a los documentos reales que responden la pregunta, y el vecindario del embedding captura eso.

```
HyDE(q):
  h ← LLM("escribe un pasaje que responda: " + q)   # hipotético, puede ser falso
  v ← E(h)                                          # embedding del pasaje, no de q
  return top-k del índice para v                    # documentos reales similares a h
```

🧩 Descomposición y step-back

- **Descomposición**: una pregunta multi-hop ("¿qué director dirigió la película más taquillera del estudio que produjo Alien?") se divide en sub-preguntas secuenciales, cada una recuperable por separado; la respuesta de una alimenta la siguiente. Es la aplicación a retrieval del *least-to-most prompting* (arXiv:2205.10625).
- **Step-back prompting** (arXiv:2310.06117): antes de la pregunta específica se formula su versión más general ("¿qué principios regulan X?") y se recupera para ambas; el contexto de principios generales mejora el razonamiento sobre el caso concreto.

🚦 Enrutamiento (routing)

Con varias fuentes (índice vectorial, base SQL, grafo, API), un **router** —un clasificador o un LLM con salida estructurada— decide a qué fuente(s) va cada consulta y con qué técnica. El router es un punto único de fallo: si clasifica mal, todo lo posterior recupera en el lugar equivocado.

🧮 Ejemplo trabajado

Descomposición de una consulta multi-hop:

```
Q: "¿Qué edad tenía el fundador de la empresa que compró Instagram
    cuando lanzó su primer producto?"

Paso 1: "¿Qué empresa compró Instagram?"          → recupera → "Facebook (2012)"
Paso 2: "¿Quién fundó Facebook?"                  → recupera → "Mark Zuckerberg"
Paso 3: "¿Cuándo lanzó Zuckerberg su primer producto (Facebook)?" → "2004"
Paso 4: "¿En qué año nació Zuckerberg?"           → recupera → "1984"
Síntesis: 2004 − 1984 = 20 años.
```

Con la consulta original sin descomponer, el retriever busca un único pasaje que contenga *toda* la cadena — que probablemente no existe en el corpus. La descomposición convierte una pregunta sin documento soporte en cuatro preguntas con soporte directo. El coste: 4 recuperaciones + 4 llamadas al LLM, y los errores se **propagan** — si el paso 1 devuelve la empresa equivocada, todo lo demás es coherentemente erróneo.

📊 Propiedades y comparación

Técnica	Ataca	Coste extra	Riesgo	Cuándo usarla
Rewriting	consultas mal formuladas / correferencia	1 llamada LLM	sobre-reescribir la intención	conversacional, siempre barato
Multi-query + RRF	vocabulario / recall	m llamadas + m búsquedas	variantes redundantes	recall bajo con consulta única
HyDE	asimetría pregunta-documento	1 llamada + 1 búsqueda	el hipotético desvía el tema	zero-shot, corpus técnico
Descomposición	preguntas multi-hop	n pasos secuenciales	propagación de errores	la respuesta cruza documentos
Step-back	falta de contexto general	1 llamada + 2 búsquedas	contexto genérico irrelevante	preguntas específicas con principios detrás
Routing	múltiples fuentes	1 clasificación	enrutar mal (fallo total)	arquitecturas con varias fuentes

⚠️ Errores conceptuales frecuentes

1. **"HyDE necesita que el LLM sepa la respuesta"**. No: el documento hipotético puede ser factualmente falso y funcionar, porque lo que se aprovecha es su forma y vocabulario, no su contenido. Falla cuando el hipotético se desvía de tema, no cuando se equivoca en datos.
2. **Aplicar todas las técnicas a la vez**. Cada capa añade latencia, coste y varianza. Se parte de la búsqueda directa medida (baseline) y se añade una técnica solo si una métrica lo justifica.

3. **Descomponer preguntas simples**. Una pregunta de un salto descompuesta en tres sub-preguntas multiplica el coste y las oportunidades de error sin ganancia.
4. **Ignorar la propagación de errores**. En descomposición secuencial el error del paso 1 contamina todo; los pasos deben validar sus premisas o el sistema debe poder retroceder.
5. **Router opaco**. Si no se registra qué ruta tomó cada consulta, los fallos de recuperación son indiagnosticables: no se sabe si falló la técnica o la elección de técnica.

🚀 Del aprendizaje a la operación

Entre este núcleo y una capa de consultas operativa faltan: un conjunto de consultas reales etiquetadas para medir qué técnica aporta y a qué coste (la mayoría de las consultas de producción son simples y no necesitan nada), presupuestos de latencia por ruta (multi-query y descomposición multiplican llamadas), *fallbacks* cuando el LLM reescritor falla o devuelve formato inválido, y registro por consulta de la ruta elegida y las sub-consultas generadas para poder auditar y mejorar el router con datos.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Gao, L. et al. (2022). *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE). arXiv:2212.10496 — uso: fuente primaria del mecanismo estudiado
- Ma, X. et al. (2023). *Query Rewriting for Retrieval-Augmented Large Language Models*. arXiv:2305.14283 — uso: fuente primaria del mecanismo estudiado
- Zheng, H. et al. (2023). *Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models*. arXiv:2310.06117 — uso: fuente primaria del mecanismo estudiado
- Zhou, D. et al. (2022). *Least-to-Most Prompting Enables Complex Reasoning in Large Language Models*. arXiv:2205.10625 — uso: fuente primaria del mecanismo estudiado
- Cormack, G., Clarke, C. & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. SIGIR '09. DOI 10.1145/1571941.1572114 — uso: fuente primaria del mecanismo estudiado
- Documentación de LangChain, *MultiQueryRetriever*: https://python.langchain.com/docs/how_to/MultiQueryRetriever/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define transformación y descomposición de consultas sin usar una marca o framework como definición.
2. Explica la relación entre query rewrite, decomposition, routing, retrieval.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

107 — Knowledge graphs y GraphRAG

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **knowledge graphs y graphrag** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar knowledge graphs y graphrag usando los conceptos `knowledge graph`, `entidades`, `relaciones`, `GraphRAG`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`knowledge graph`, `entidades`, `relaciones`, `GraphRAG`

🗺️ Ubicación en el mapa de la IA

Los knowledge graphs son la herencia de la IA simbólica (redes semánticas, lógica de descripción, la Web Semántica) dentro de la era neuronal: conocimiento como **entidades y relaciones explícitas** en vez de vectores opacos. El RAG vectorial (097-103) recupera pasajes similares; falla cuando la respuesta exige **conectar** hechos dispersos o **agregar** sobre todo el corpus. GraphRAG (Microsoft, 2024) cierra el círculo: usa LLMs para construir el grafo y el grafo para estructurar lo que el LLM lee — neuro-simbólico en la práctica.

📖 Fundamentos

🔺 Tripletas: la unidad del conocimiento explícito

Un **knowledge graph** representa el conocimiento como tripletas **(sujeto, predicado, objeto)**: nodos = entidades, aristas = relaciones tipadas y dirigidas. En el estándar RDF del W3C toda afirmación tiene esa forma; en los **grafos de propiedades** (Neo4j) nodos y aristas llevan además atributos clave-valor (`fecha`, `fuente`, `confianza`). Frente al texto, el grafo aporta:

- **Composicionalidad**: los hechos se conectan por entidades compartidas; responder "multi-hop" es recorrer caminos, no esperar que exista un pasaje con toda la cadena.
- **Agregación**: contar, agrupar y resumir sobre relaciones ("todos los proveedores de X") es una consulta, no una lectura de todo el corpus.
- **Procedencia por hecho**: cada tripleta puede llevar su fuente, no solo cada documento.

🔎 Consultar el grafo: Cypher conceptual

Cypher (Neo4j) expresa patrones de caminos con sintaxis "ASCII-art": `(nodo)` y `-[relación]->`. No hace falta memorizarla; hace falta leer el patrón:

```
MATCH (p:Persona)-[:FUNDÓ]->(e:Empresa)-[:ADQUIRIÓ]->(x:Empresa {nombre: "Instagram"})
RETURN p.nombre
```

"Encuentra la persona que fundó la empresa que adquirió Instagram": los dos saltos que en la clase 106 exigían descomposición secuencial son aquí **un solo patrón declarativo** resuelto por el motor del grafo.

🧠 Extracción de tripletas con LLM

Construir el grafo a mano no escala; los LLMs extraen tripletas de texto con un prompt de extracción (entidades → relaciones → normalización). Los dos problemas duros no son la extracción sino después:

- **Resolución de entidades**: "IBM", "International Business Machines" y "la compañía" deben colapsar en un solo nodo, o el grafo se fragmenta.
- **Normalización de relaciones**: "fundó", "creó", "estableció" deben mapear a un esquema finito de predicados, o cada hecho queda en su propio dialecto.

🌐 GraphRAG (Microsoft, arXiv:2404.16130)

GraphRAG ataca las preguntas **globales** ("¿cuáles son los temas dominantes en este corpus?") que el RAG vectorial no puede responder — ningún top-k local las cubre. Pipeline de indexación:

```
1. Extraer entidades y relaciones de cada chunk con un LLM  → grafo del corpus
2. Detectar comunidades jerárquicas (algoritmo de Leiden)   → clusters de entidades
3. Resumir cada comunidad con un LLM                        → resúmenes por nivel
Consulta global: map-reduce sobre resúmenes de comunidades → respuesta agregada
Consulta local:  entidades de la consulta → vecindario del grafo + chunks asociados
```

El grafo funciona como un índice temático jerárquico construido una vez (coste alto de indexación en llamadas LLM) y consultado muchas veces.

🧮 Ejemplo trabajado

Extracción de tripletas del párrafo:

> "Marie Curie descubrió el polonio en 1898 junto a su esposo Pierre. Por sus investigaciones sobre la radiactividad recibió el Premio Nobel de Física en 1903, compartido con Pierre y Henri Becquerel."

```
(Marie Curie,  DESCUBRIÓ,       polonio)        {año: 1898}
(Pierre Curie, DESCUBRIÓ,       polonio)        {año: 1898}   ← "junto a su esposo"
(Marie Curie,  CASADA_CON,      Pierre Curie)
(Marie Curie,  RECIBIÓ,         Nobel de Física) {año: 1903}
(Pierre Curie, RECIBIÓ,         Nobel de Física) {año: 1903}
(H. Becquerel, RECIBIÓ,         Nobel de Física) {año: 1903}
(Marie Curie,  INVESTIGÓ,       radiactividad)
```

Decisiones no triviales que el ejemplo esconde: "su esposo Pierre" exige resolver la correferencia al nodo `Pierre Curie`; el "compartido con" genera tres tripletas `RECIBIÓ` distintas, no una; y el año va como **propiedad** de la arista, no como nodo. Con el grafo, "¿quién compartió un Nobel con un descubridor del polonio?" es un patrón de dos saltos; en texto plano exigiría reunir frases dispersas.

📊 Propiedades y comparación

Dimensión	RAG vectorial	GraphRAG / KG
Unidad recuperada	chunk de texto	entidades, caminos, resúmenes de comunidad
Preguntas multi-hop	frágil (requiere descomposición)	naturales (patrón de camino)
Preguntas globales/agregadas	no alcanzables por top-k	resúmenes jerárquicos (map-reduce)
Coste de indexación	embeddings: bajo	extracción LLM + comunidades: alto
Actualización	añadir chunk e indexar	insertar tripletas + rehacer comunidades afectadas
Fallos típicos	contexto irrelevante	entidades sin resolver, esquema inconsistente
Procedencia	por chunk	por hecho (tripleta)

⚠️ Errores conceptuales frecuentes

1. **"El grafo reemplaza al índice vectorial"**. Son complementarios: el grafo responde estructura y agregación; el vectorial, similitud semántica sobre texto libre. GraphRAG usa ambos (búsqueda local parte de embeddings de entidades).
2. **Confundir esquema con datos**. Definir predicados (`FUNDÓ`, `ADQUIRIÓ`) es diseño de esquema; poblarlos es extracción. Saltarse el esquema produce un grafo donde cada hecho usa su propio predicado y nada conecta.
3. **Asumir que la extracción LLM es fiable**. Las tripletas extraídas heredan las alucinaciones y omisiones del modelo; sin muestreo de verificación contra el texto fuente, el grafo institucionaliza errores con apariencia de base de datos.

4. **Ignorar la resolución de entidades**. Es el fallo silencioso más común: el grafo "funciona" pero cada mención es una isla y los caminos multi-hop no existen.
5. **Usar GraphRAG para preguntas locales simples**. Si la respuesta vive en un pasaje, el RAG vectorial la encuentra por una fracción del coste; GraphRAG se justifica en preguntas globales o relacionales.

🚀 Del aprendizaje a la operación

Entre este núcleo y un GraphRAG operativo faltan: el coste real de indexación (miles de llamadas LLM por corpus mediano, que se repiten al re-extraer), un proceso de calidad para la resolución de entidades (muestreo, métricas de duplicación), el versionado del esquema de predicados cuando el dominio evoluciona, la actualización incremental de comunidades sin reconstruir el grafo entero, y la decisión operativa de qué preguntas enrutar al grafo y cuáles al índice vectorial (clase 106, routing).

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Edge, D. et al. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv:2404.16130 — uso: fuente primaria del mecanismo estudiado
- Hogan, A. et al. (2021). *Knowledge Graphs*. ACM Computing Surveys. arXiv:2003.02320 — uso: fuente primaria del mecanismo estudiado
- Traag, V., Waltman, L. & van Eck, N. (2019). *From Louvain to Leiden: guaranteeing well-connected communities*. arXiv:1810.08473 — uso: fuente primaria del mecanismo estudiado
- Documentación oficial de Microsoft GraphRAG: https://microsoft.github.io/graphrag/ — uso: referencia consultada en su fuente original
- W3C (2014). *RDF 1.1 Concepts and Abstract Syntax*: https://www.w3.org/TR/rdf11-concepts/ — uso: marco normativo de referencia
- Documentación de Cypher (Neo4j): https://neo4j.com/docs/cypher-manual/current/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P31 · Agentes generativos: simulacros interactivos de comportamiento humano	2023	Resuelve la memoria de un agente que vive mucho tiempo: qué recordar, cuándo y por qué, cuando el contexto no da para todo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

Normas y documentación oficial que aplica esta clase: https://www.w3.org/TR/rdf11-concepts/

📝 Evaluación completa

❓ Preguntas

1. Define knowledge graphs y graphrag sin usar una marca o framework como definición.
2. Explica la relación entre knowledge graph, entidades, relaciones, GraphRAG.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

108 — Memoria de corto y largo plazo

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `agent` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **memoria de corto y largo plazo** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar memoria de corto y largo plazo usando los conceptos `memory`, `thread`, `store`, `episodic`, `semantic`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`memory`, `thread`, `store`, `episodic`, `semantic`

🗺️ Ubicación en el mapa de la IA

Un LLM es amnésico por construcción: cada llamada parte de cero y solo "recuerda" lo que cabe en su ventana de contexto. La memoria de los sistemas de IA no está en el modelo sino **alrededor** de él: es ingeniería de recuperación aplicada al propio historial del sistema. Esta clase toma la taxonomía de la psicología cognitiva (Tulving: memoria episódica, semántica, procedimental), la traduce a arquitectura de software, y prepara el terreno de los agentes con estado persistente (parte 09) — Generative Agents y MemGPT demostraron que la gestión de memoria es lo que separa un chatbot de un agente coherente en el tiempo.

📖 Fundamentos

⏱️ Memoria de corto plazo: la ventana de contexto

La memoria de corto plazo de un sistema LLM es el **hilo actual** (*thread*): los mensajes que se reenvían en cada llamada. Propiedades: capacidad finita (la ventana), coste lineal en tokens por llamada, y desaparece al cerrar el hilo. Gestionarla es decidir **qué se reenvía**: todo el historial (caro y eventualmente imposible), una

ventana deslizante de los últimos n turnos (barato, olvida lo antiguo), o un resumen acumulado más los turnos recientes.

🗄️ Memoria de largo plazo: el store

La memoria de largo plazo es un **almacén externo** (*store*) que sobrevive a los hilos, con dos operaciones: **escribir** (qué merece persistir, con qué esquema) y **leer** (recuperar lo relevante para el turno actual — exactamente el retrieval de las clases 097-101, aplicado sobre los recuerdos). Taxonomía funcional, heredada de la psicología cognitiva:

- **Episódica** — eventos con tiempo y contexto: "el 12 de mayo el usuario reportó el bug X y lo resolvimos con Y". Soporta preguntas sobre el pasado y aprendizaje por ejemplos (few-shot desde experiencias propias).
- **Semántica** — hechos estables destilados de los episodios: "el usuario prefiere respuestas breves", "el proyecto usa PostgreSQL". Es la que se consulta en casi todos los turnos; se representa como perfil estructurado o colección de hechos indexados.
- **Procedimental** — cómo actuar: las instrucciones del sistema, reglas aprendidas ("nunca desplegar en viernes"). Modificarla cambia el comportamiento en todos los hilos futuros; es la más delicada.

🔁 Compactación: de episodios a semántica

El puente entre corto y largo plazo es la **compactación** (*consolidación*): cuando el hilo crece o termina, un LLM resume lo ocurrido, extrae hechos estables y los escribe al store. Decisiones críticas:

```
compactar(hilo):
  resumen   ← LLM("resume decisiones, hechos y pendientes": hilo)
  hechos    ← LLM("extrae hechos estables sobre el usuario/dominio": hilo)
  para cada hecho:
      si contradice el store → política: ¿sobrescribir, versionar, preguntar?
      si no → upsert(store, hecho, {fuente: hilo_id, fecha})
  hilo ← [resumen] + últimos_n_turnos          # el hilo se acorta, no se pierde todo
```

- La compactación es **con pérdida**: lo que el resumen omite deja de existir para el sistema. Qué se pierde es una decisión de diseño, no un accidente.
- El **olvido** es una función, no un fallo: memoria que caduca (TTL), se sobrescribe (el hecho nuevo reemplaza al viejo) o decae por falta de uso evita que el store acumule contradicciones y ruido.

📏 Memory engineering: de heurística a disciplina medible

Hacia 2026 la memoria de agentes dejó de ser un cajón de heurísticas y se convirtió en **memory engineering**: una práctica con benchmarks estandarizados que permiten comparar arquitecturas de memoria *distintas* (grafo de hechos, vector store, resumen jerárquico, archivos planos) sobre el mismo conjunto de evaluación — preguntas cuya respuesta exige recordar información de sesiones anteriores, actualizar hechos que cambiaron y abstenerse cuando el recuerdo no existe (p. ej. LongMemEval). La consecuencia práctica para esta clase: cualquier decisión de diseño del store (esquema, política de olvido, umbral de compactación) debe justificarse con una métrica sobre un conjunto de recuerdo, no con la intuición de que "recordar más es mejor" — recordar de más introduce contradicciones y contexto muerto que degradan la respuesta tanto como olvidar de menos.

🧮 Ejemplo trabajado

Hilo de soporte técnico con presupuesto de contexto de 4 000 tokens:

```
Estado: 12 turnos, ~5 200 tokens → excede el presupuesto.

Compactación:
  resumen (180 tokens): "Usuaria Ana, error 502 en el despliegue del servicio pagos
    tras actualizar a v2.3.1; causa: variable DB_POOL sin migrar; pendiente: verificar
    en staging."
  hechos → memoria semántica:
    {usuario: Ana, rol: DevOps}                    {fuente: hilo-88, 2026-07-30}
    {servicio: pagos, versión: v2.3.1}             {fuente: hilo-88, 2026-07-30}
  episodio → memoria episódica:
    "2026-07-30: error 502 resuelto migrando DB_POOL"

Hilo nuevo = resumen (183) + últimos 4 turnos (~900) ≈ 1 080 tokens  (de 5 200)

Turno siguiente: "¿me confirmas lo de staging?"
  lectura del store: nada nuevo necesario — el pendiente está en el resumen → responde.
Tres semanas después, otro hilo: "otra vez un 502 en pagos"
  lectura episódica por similitud: recupera el episodio del 30-07 → propone revisar
  DB_POOL antes de diagnosticar de cero.
```

La compresión fue 5 200 → 1 080 tokens (~79 %) y el sistema conservó exactamente lo que las consultas posteriores necesitaron. Si el resumen hubiera omitido "pendiente: staging", la primera pregunta habría fallado: la calidad de la compactación **es** la calidad de la memoria.

📊 Propiedades y comparación

Dimensión	Corto plazo (thread)	Episódica (store)	Semántica (store)	Procedimental
Contenido	turnos del hilo actual	eventos con fecha y contexto	hechos estables destilados	reglas e instrucciones
Alcance	un hilo	entre hilos	entre hilos	global
Escritura	automática (append)	al cerrar hilo / por evento	por compactación o explícita	deliberada, con revisión
Lectura	se reenvía entera/resumida	por similitud o fecha	en casi cada turno	en cada llamada (system)
Riesgo dominante	desbordar la ventana	crecer sin límite	hechos obsoletos o contradictorios	corromper el comportamiento global

⚠️ Errores conceptuales frecuentes

1. **"El modelo recuerda la conversación"**. No: el modelo es sin estado; recuerda el *sistema* que reenvía el historial en cada llamada. Confundirlos lleva a "memorias" que desaparecen al cambiar de hilo.
2. **"Contexto más largo elimina la necesidad de memoria"**. Una ventana de 1M tokens sigue siendo un hilo: cara por llamada, sin persistencia entre hilos, y con atención degradada en el medio (arXiv:2307.03172). La memoria de largo plazo es selección, no capacidad.
3. **Guardarlo todo**. Un store sin criterio de escritura ni olvido acumula hechos obsoletos y contradicciones; la recuperación devuelve ruido con confianza. Escribir poco y estable supera a escribir todo.
4. **Compactar sin política de conflictos**. Si el hecho nuevo ("prefiere respuestas detalladas") contradice al almacenado ("prefiere brevedad") y el upsert sobrescribe a ciegas, la memoria oscila con el último humor del usuario.
5. **Tratar la memoria procedimental como una más**. Un hecho episódico erróneo afecta una respuesta; una regla procedimental errónea corrompe todos los hilos futuros. Su escritura exige revisión (humana o con validación) proporcional a ese radio de daño.

🚀 Del aprendizaje a la operación

Entre este núcleo y una memoria en producción faltan: aislamiento por usuario y por tenant (la memoria de uno no puede filtrarse al hilo de otro), cumplimiento del derecho de supresión (borrar de verdad, incluidos los resúmenes derivados), cifrado y control de acceso del store, evaluación de la compactación (¿qué preguntas posteriores fallan por información perdida?), límites de crecimiento con TTL y decaimiento, y trazabilidad de qué recuerdo influyó en qué respuesta — sin eso, depurar un agente con memoria es imposible.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("agent")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Park, J. S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. arXiv:2304.03442 — uso: fuente primaria del mecanismo estudiado
- Packer, C. et al. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560 — uso: fuente primaria del mecanismo estudiado
- Liu, N. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172 — uso: fuente primaria del mecanismo estudiado
- Documentación de LangGraph, *Memory*: https://langchain-ai.github.io/langgraph/concepts/memory/ — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. *Artificial Intelligence: A Modern Approach* (4.ª ed.), cap. 2 (agentes y estado interno). https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Wu, D. et al. (2024). *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. arXiv:2410.10813 — uso: fuente primaria del mecanismo estudiado
- Memo — *State of AI Agent Memory 2026* (panorama de benchmarks y arquitecturas de memoria): https://memo.ai/blog/state-of-ai-agent-memory-2026 — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P37 · MemGPT: modelos de lenguaje como sistemas operativos	2023	Aplica al contexto la idea de memoria virtual: una jerarquía que da la ilusión de memoria grande sobre una pequeña y rápida.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 2
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define memoria de corto y largo plazo sin usar una marca o framework como definición.
2. Explica la relación entre memory, thread, store, episodic, semantic.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

109 — Compresión de contexto y cachés semánticos

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `retrieval` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **compresión de contexto y cachés semánticos** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar compresión de contexto y cachés semánticos usando los conceptos `compression`, `cache`, `context`, `budget`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`compression`, `cache`, `context`, `budget`

🗺️ Ubicación en el mapa de la IA

Cuando RAG y memoria (102-105) funcionan, el problema pasa a ser económico: cada token del contexto cuesta dinero y latencia en **cada** llamada, y los pipelines maduros repiten una enorme fracción de su contexto (instrucciones, documentos, historial). Esta clase aplica dos ideas clásicas de sistemas —compresión y caché— al contexto de los LLMs. Son las técnicas que separan un prototipo que funciona de un servicio cuyo coste por consulta permite operarlo, y preparan la evaluación coste/calidad de las clases 107-108.

📖 Fundamentos

💰 El presupuesto de contexto

Cada llamada paga `O(tokens de entrada)` en coste y latencia (el *prefill* procesa todo el prompt antes del primer token de salida). Un **presupuesto de contexto** reparte la ventana entre partes con valor desigual:

```
contexto = instrucciones (fijas) + herramientas + memoria + top-k recuperado + turno
prioridad: lo que cambia la respuesta > lo que la decora
```

La pregunta de diseño no es "¿cuánto cabe?" sino "¿qué aporta cada token a la calidad de la respuesta?" — y medirla (clase 110) es lo único que legitima recortar. Esta pregunta es el núcleo de lo que la industria llama hoy **context engineering** (el menor conjunto de tokens de alta señal por llamada); la clase 118 la retoma para el caso agéntico, donde el contexto además crece con cada iteración del bucle.

Compresión de contexto

Dos familias:

- **Compresión dura (extractiva)**: eliminar tokens de baja información conservando los demás. **LLMLingua** (arXiv:2310.05736) usa un modelo de lenguaje pequeño para estimar la perplejidad de cada token y descarta los más predecibles (artículos, redundancias): ratios de 2-10× con degradación pequeña en tareas de QA. **LongLLMLingua** (arXiv:2310.06839) añade compresión guiada por la pregunta: conserva lo relevante *para esta consulta*.
- **Compresión blanda (abstractiva)**: resumir con un LLM (la compactación de la clase 105 es el caso conversacional). Más fluida, pero puede parafrasear mal un dato crítico y es más cara de producir.

En RAG, comprimir los pasajes recuperados ataca además el problema "lost in the middle" (arXiv:2307.03172): menos tokens irrelevantes entre la pregunta y la evidencia.

Prompt caching (caché exacto de prefijos)

Los proveedores de LLM cachean el **estado de atención (KV-cache) de un prefijo exacto** del prompt: si la siguiente llamada comparte ese prefijo token a token, el prefill se salta esa parte (en la API de Anthropic, la lectura de caché cuesta ~10 % del token normal). Consecuencia arquitectónica directa: el prompt se ordena de **estable a volátil** — instrucciones y herramientas primero, documentos después, el turno del usuario al final. Un solo token cambiado invalida todo lo que le sigue.

Caché semántico (aproximado, por similitud)

Un **caché semántico** reutiliza la **respuesta final** cuando llega una consulta *equivalente*, no idéntica: se embebe la consulta (clase 100) y se busca en el caché por similitud; si `sim ≥ τ`, se devuelve la respuesta almacenada sin llamar al LLM (GPTCache implementa este patrón).

```
consulta q:
  v ← E(q);  (q', r', s) ← vecino más cercano en caché
  si s ≥ τ:  return r'                # HIT: coste ≈ un embedding
  si no:     r ← pipeline_completo(q); insertar (q, r); return r
```

El umbral `τ` gobierna el trade-off: bajo → más aciertos pero **falsos aciertos** (responder otra pregunta, el peor fallo posible); alto → caché casi inútil. Además exige **invalidación**: si el corpus cambió, las respuestas cacheadas quedan obsoletas aunque la consulta sea idéntica.

Ejemplo trabajado

Caché semántico con `τ = 0.92`. En caché: `q' = "¿cómo reinicio mi contraseña?"` con su respuesta `r'`.

```
Llegan tres consultas (similitud coseno con q'):
  q1 "¿cómo restablezco mi contraseña?"        sim = 0.96 ≥ 0.92 → HIT correcto
  q2 "¿cómo cambio mi contraseña?"             sim = 0.93 ≥ 0.92 → HIT ¿correcto?
  q3 "¿cómo reinicio mi router?"               sim = 0.85 < 0.92 → MISS → pipeline

q2 es el caso frontera: "cambiar" (conociendo la actual) y "restablecer" (olvidada)
pueden tener procedimientos distintos → falso acierto potencial que τ = 0.92 no detecta.

Economía con 10 000 consultas/día, 40 % hit-rate, 0.9 s y $0.004 por llamada LLM:
  ahorro diario ≈ 10 000 · 0.40 · $0.004 = $16/día  (~$480/mes)
  latencia en hit: ~50 ms (embedding + búsqueda) frente a ~900 ms.
Coste del error: si el 2 % de los hits son falsos → 80 respuestas equivocadas/día.
¿Vale $480/mes ese riesgo? Esa es la decisión real, y depende del dominio.
```

📊 Propiedades y comparación

Técnica	Qué reutiliza/reduce	Ahorro típico	Condición de validez	Riesgo principal
Presupuesto de contexto	tokens de entrada	proporcional al recorte	medir calidad tras recortar	recortar señal, no ruido
LLMLingua (dura)	tokens poco informativos	2-10× en contexto	tarea tolerante a texto no fluido	perder el dato crítico
Resumen (blanda)	historial/documentos	variable	el resumen preserva lo consultado	paráfrasis errónea
Prompt caching	prefill de prefijo exacto	~90 % del coste del prefijo	prefijo idéntico token a token	ordenar mal el prompt
Caché semántico	la llamada completa	hit-rate × coste por llamada	`sim ≥ τ` implica misma intención	falso acierto

⚠️ Errores conceptuales frecuentes

1. **Confundir prompt caching con caché semántico**. El primero es exacto, por prefijo, ahorra *prefill* y lo gestiona el proveedor; el segundo es aproximado, por similitud, ahorra la llamada entera y lo gestiona tu sistema. Resuelven problemas distintos y conviven.
2. **"Comprimir no pierde información"**. Toda compresión de contexto es con pérdida; la pregunta es si lo perdido afectaba la respuesta, y eso solo lo dice una evaluación antes/después (clase 110).
3. **Elegir τ sin medir falsos aciertos**. Un hit-rate de 60 % es inútil si incluye 5 % de respuestas a otra pregunta; τ se calibra sobre pares etiquetados (equivalente/no equivalente), no a ojo.

4. **Cachear sin invalidación**. Corpus actualizado + caché viejo = respuestas obsoletas servidas con confianza y a máxima velocidad.
5. **Poner lo volátil al principio del prompt**. Un timestamp o el nombre del usuario en la primera línea invalida el KV-cache de todo lo que sigue en cada llamada.

🚀 Del aprendizaje a la operación

Entre este núcleo y una capa de eficiencia operativa faltan: métricas por segmento (hit-rate, tasa de falsos aciertos muestreada por humanos, ahorro neto tras el coste de embeddings y almacenamiento), invalidación conectada al pipeline de ingesta (qué entradas del caché tocan qué documentos), aislamiento del caché por usuario/tenant cuando las respuestas dependen de permisos, calibración periódica de τ con deriva de consultas, y pruebas de regresión de calidad cada vez que se ajusta el ratio de compresión.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("retrieval")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Jiang, H. et al. (2023). *LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models*. arXiv:2310.05736 — uso: fuente primaria del mecanismo estudiado
- Jiang, H. et al. (2023). *LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression*. arXiv:2310.06839 — uso: fuente primaria del mecanismo estudiado
- Liu, N. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. arXiv:2307.03172 — uso: fuente primaria del mecanismo estudiado
- Documentación de Anthropic, *Prompt caching*: https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching — uso: referencia consultada en su fuente original
- Documentación de GPTCache: https://gptcache.readthedocs.io/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P36 · Perdidos en el medio: cómo usan los modelos de lenguaje los contextos largos	2023	Tener contexto largo no es usarlo: el rendimiento cae en forma de U cuando el dato relevante está en el medio.	notebook
P37 · MemGPT: modelos de lenguaje como sistemas operativos	2023	Aplica al contexto la idea de memoria virtual: una jerarquía que da la ilusión de memoria grande sobre una pequeña y rápida.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define compresión de contexto y cachés semánticos sin usar una marca o framework como definición.
2. Explica la relación entre compression, cache, context, budget.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

110 — Evaluación de fidelidad, cobertura y atribución

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 6
Laboratorio: `evaluation` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **evaluación de fidelidad, cobertura y atribución** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar evaluación de fidelidad, cobertura y atribución usando los conceptos `faithfulness`, `recall`, `attribution`, `citations`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`faithfulness`, `recall`, `attribution`, `citations`

🗺️ Ubicación en el mapa de la IA

Un pipeline RAG tiene dos componentes que fallan de formas distintas: el retriever puede traer contexto equivocado y el generador puede desviarse del contexto correcto. La evaluación clásica de IR (precision/recall, clase 102) mide lo primero; la evaluación de generación clásica (BLEU, ROUGE) no mide lo segundo. Esta clase presenta el marco que llenó ese hueco —métricas de fidelidad y atribución, con RAGAS como referencia — y el patrón *LLM-as-judge* que las hace computables sin anotación humana exhaustiva. Es el prerrequisito del proyecto auditable de la clase 111: sin estas métricas, "funciona bien" es una opinión.

📖 Fundamentos

🧭 Qué se evalúa y contra qué

Una interacción RAG tiene cuatro elementos: pregunta `q`, contexto recuperado `C`, respuesta generada `a`, y (en evaluación) la referencia *ground truth* `g`. Cada métrica compara un par distinto — confundirlos es el error más común:

Métrica (RAGAS, arXiv:2309.15217)	Compara	Pregunta que responde	¿Necesita ground truth?
Faithfulness	`a` vs `C`	¿la respuesta se sostiene en el contexto?	no
Answer relevancy	`a` vs `q`	¿responde a lo que se preguntó?	no
Context precision	`C` vs `q` / `g`	¿lo recuperado es relevante (y bien ordenado)?	idealmente
Context recall	`C` vs `g`	¿el contexto cubre lo necesario para responder?	sí

📏 Fidelidad (faithfulness)

Se computa en dos pasos con un LLM evaluador (*LLM-as-judge*):

```
1. Descomponer a en afirmaciones atómicas s1..sn   (claims)
2. Para cada si: ¿C la implica? (entailment sí/no)
faithfulness = |afirmaciones implicadas| / n
```

Fidelidad baja = alucinación **respecto al contexto** (aunque el dato sea cierto en el mundo). Es la métrica que no requiere referencia y por eso puede monitorearse en producción sobre tráfico real.

🎯 Cobertura del retriever (context precision / recall)

- **Context recall**: fracción de las afirmaciones de la referencia `g` que el contexto recuperado respalda. Recall bajo = el retriever no trajo lo necesario; ningún generador lo compensa (clase 104: re-rankear no crea recall).
- **Context precision**: fracción del contexto que es realmente relevante, ponderando el orden (los pasajes útiles deben estar arriba). Precision baja = ruido que paga tokens y degrada la atención.

🔗 Atribución (citas verificables)

La atribución evalúa el vínculo afirmación→cita. El marco AIS (arXiv:2112.12870) define el criterio riguroso: una cita es válida si el pasaje citado **implica** la afirmación ("attributable to identified sources"). Sobre respuestas con citas (clase 105) se miden, siguiendo ALCE (arXiv:2305.14627):

- **Citation recall**: ¿cada afirmación tiene alguna cita que la respalda?
- **Citation precision**: ¿cada cita emitida respalda de verdad su afirmación?

⚖️ LLM-as-judge: potencia y sospecha

Usar un LLM para juzgar entailment escala a miles de ejemplos, pero el juez tiene sesgos conocidos (posición, verbosidad, autopreferencia) y varianza. Regla del programa: el juez se **calibra** contra una muestra anotada por humanos (¿qué % de acuerdo?) antes de confiar en sus números, y se re-calibra al cambiar de modelo juez o de dominio.

🧮 Ejemplo trabajado

Caso completo, calculado a mano:

```
q: "¿Cuándo se inauguró el museo y quién lo diseñó?"
C: [1] "El museo se inauguró en octubre de 1997."
   [2] "El edificio fue diseñado por Frank Gehry."
g: "Se inauguró en octubre de 1997 y lo diseñó Frank Gehry."

a: "El museo, diseñado por Frank Gehry [2], se inauguró en 1997 [1] y recibió
    un millón de visitantes en su primer año."

Afirmaciones de a:      ¿implicada por C?    ¿cita correcta?
  s1 diseñado por Gehry        SÍ            [2] la respalda → ✓
  s2 se inauguró en 1997       SÍ            [1] la respalda → ✓
  s3 un millón de visitantes   NO            sin cita        → ✗

faithfulness      = 2/3 ≈ 0.67     (s3 no está en el contexto)
citation recall   = 2/3 ≈ 0.67     (s3 carece de cita que la respalde)
citation precision= 2/2 = 1.00     (las citas emitidas son correctas)
context recall    = 2/2 = 1.00     (todo lo que g necesita está en C)
context precision = 2/2 = 1.00     (ambos pasajes son relevantes)
```

Diagnóstico: el retriever hizo su trabajo perfecto; el fallo es exclusivamente del generador (s3 salió de la memoria paramétrica). Sin descomponer por métrica, solo se vería "la respuesta tiene algo mal" sin saber qué componente arreglar.

📊 Propiedades y comparación

La tabla de la sección de fundamentos resume qué compara cada métrica; el diagrama muestra la propiedad clave del marco: **cada métrica apunta a un componente**, y por eso el conjunto es diagnóstico, no solo un número agregado.

⚠️ Errores conceptuales frecuentes

1. **"Fidelidad = veracidad"**. Fidelidad es coherencia con el contexto recuperado. Una respuesta puede ser fiel a un contexto erróneo (fidelidad 1.0, verdad 0) o infiel pero cierta (el modelo corrigió al corpus con su memoria — sigue siendo un fallo de RAG porque rompe la trazabilidad).
2. **Promediar todo en un solo score**. Un 0.8 global puede ser retriever perfecto + generador alucinando, o al revés; sin métricas por componente no hay acción correctiva posible.
3. **Evaluar solo con casos que tienen respuesta**. Un conjunto de evaluación sin preguntas *sin* respuesta en el corpus no mide la capacidad de rechazar — y el rechazo correcto es parte del contrato (clase 105).

4. **Confiar en el juez sin calibrarlo**. El acuerdo juez-humano varía por dominio e idioma; un juez no calibrado produce métricas precisas y sistemáticamente sesgadas.
5. **Optimizar citation recall forzando citas**. Instruir "cita siempre algo" sube el recall de citas y hunde su precisión: aparecen citas decorativas que no implican la afirmación. Las dos se reportan juntas o ninguna vale.

🚀 Del aprendizaje a la operación

Entre este núcleo y una evaluación operativa faltan: un conjunto de evaluación propio y versionado (100-500 preguntas del dominio con referencias revisadas, incluyendo preguntas sin respuesta y adversariales), la calibración documentada del juez contra anotación humana, la integración en CI (una regresión de fidelidad bloquea el despliegue igual que un test roto), el monitoreo en producción con faithfulness muestreado sobre tráfico real, y el presupuesto: evaluar con LLM-juez cuesta dinero por ejecución y ese coste condiciona cuántas veces se corre.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("evaluation")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Es, S. et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217 — uso: fuente primaria del mecanismo estudiado
- Rashkin, H. et al. (2021). *Measuring Attribution in Natural Language Generation Models* (AIS). arXiv:2112.12870 — uso: fuente primaria del mecanismo estudiado
- Gao, T. et al. (2023). *Enabling Large Language Models to Generate Text with Citations* (ALCE). arXiv:2305.14627 — uso: fuente primaria del mecanismo estudiado
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685 — uso: fuente primaria del mecanismo estudiado
- Documentación oficial de RAGAS: https://docs.ragas.io/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook
P36 · Perdidos en el medio: cómo usan los modelos de lenguaje los contextos largos	2023	Tener contexto largo no es usarlo: el rendimiento cae en forma de U cuando el dato relevante está en el medio.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

📝 Evaluación completa

❓ Preguntas

1. Define evaluación de fidelidad, cobertura y atribución sin usar una marca o framework como definición.
2. Explica la relación entre faithfulness, recall, attribution, citations.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;

- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

111 — Proyecto: RAG productivo y auditable

Parte: 08 — Recuperación, contexto, memoria y conocimiento
Nivel: avanzado · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: rag productivo y auditable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: rag productivo y auditable usando los conceptos `RAG`, `evals`, `observabilidad`, `seguridad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`RAG`, `evals`, `observabilidad`, `seguridad`

🗺️ Ubicación en el mapa de la IA

Este proyecto cierra la parte 08 integrando todas sus piezas —indexación (097-099), fusión y re-ranking (100-101), generación con citas (105), transformación de consultas (106), memoria (108), eficiencia (109) y evaluación (110)— bajo dos requisitos que las demos ignoran: que el sistema sea **operable** (medible, monitoreable, con coste conocido) y **auditable** (capaz de responder, meses después, qué evidencia exacta produjo cada respuesta). Es también el puente a la parte 09: un RAG auditable es el sustrato sobre el que se pueden construir agentes con herramientas sin perder la trazabilidad.

📖 Fundamentos

🏗️ Arquitectura de referencia

```
INGESTA:   fuentes → parsing → chunking (101) → embeddings (100) → índice vectorial
                                              → índice léxico BM25 (102)
           cada chunk con: doc_id, versión, hash, permisos, fecha

CONSULTA:  q → transformación (106) → híbrida + RRF (103) → re-rank + filtros (104)
             → prompt con citas (102, contexto comprimido 106) → LLM → respuesta [n]
             → verificación de atribución (110) → usuario

TRANSVERSAL: trazas por consulta · evals en CI · control de acceso · caché (109)
```

🔍 Observabilidad: la traza como unidad

La unidad de observabilidad de un RAG es la **traza por consulta**: un registro estructurado que encadena todo lo que produjo la respuesta. Sin ella no hay depuración (¿falló el retriever o el generador?), ni auditoría, ni datos para mejorar.

```
traza = {
  query_id, timestamp, usuario/tenant,
  consulta_original, consultas_transformadas,
  chunks_recuperados: [(chunk_id, doc_id, versión_doc, score_1a, score_rerank)],
  chunks_en_prompt, hash_del_prompt, modelo y versión, parámetros,
  respuesta, citas_emitidas, faithfulness_muestreado,
  latencias por etapa, tokens y coste
}
```

Auditable significa: dado un `query_id` de hace seis meses, poder reconstruir qué versión de qué documentos sustentó la respuesta — lo que exige versionar el corpus y el índice, no solo el código.

🛡️ Seguridad específica de RAG

El corpus es una **superficie de ataque** (OWASP LLM Top 10):

- **Inyección indirecta de prompt**: un documento ingerido contiene instrucciones ("ignora tus reglas y…") que el generador lee como contexto. Mitigación: separar estructuralmente instrucciones de datos, tratar el contexto como no confiable, filtros de salida.
- **Fuga por permisos**: el retriever encuentra documentos que el usuario no debería ver. El filtrado por ACL debe ocurrir **en la recuperación** (pre-filtro del índice), no después de generar — una cita filtrada ya es una fuga.
- **Envenenamiento del corpus**: quien pueda escribir en las fuentes escribe, indirectamente, en las respuestas. La ingesta necesita control de procedencia.
- **Exfiltración vía citas o logs**: las trazas contienen fragmentos de documentos; heredan la clasificación de seguridad del corpus.

🔁 Evaluación como puerta de despliegue

El conjunto de evaluación (110) se ejecuta en CI: cada cambio —de prompt, de modelo, de chunking, de umbral— pasa por las métricas antes de desplegarse. Reglas mínimas: umbrales de no-regresión (faithfulness y context recall no bajan), presupuesto (coste y latencia p95 no suben más de X %), y casos de rechazo obligatorios (las preguntas sin respuesta se siguen rechazando). En producción: muestreo continuo de faithfulness + feedback de usuarios como señal de deriva.

🧮 Ejemplo trabajado

Auditoría de un incidente con la traza: un usuario reporta (día 90) que el día 12 el sistema respondió "el límite de gasto es 5 000 €" y el límite real era 3 000 €.

```
1. buscar query_id por usuario+fecha        → q-4471
2. traza[q-4471].chunks_en_prompt           → chunk c-812 de politica_gastos.md v7
3. traza[q-4471].citas                      → la respuesta citó [1] = c-812  ✓ atribución correcta
4. corpus_versionado(politica_gastos.md)    → v7 (vigente el día 12) decía 5 000 €;
                                              v8 (día 30) lo bajó a 3 000 €
Veredicto: el sistema fue FIEL a la versión vigente; el fallo fue de actualización
de la política, no del RAG. Acción: ninguna en el pipeline; el hallazgo es del proceso
documental.

Contraescenario: si chunks_en_prompt hubiera mostrado v8 y la respuesta 5 000 €,
el fallo sería del generador (infidelidad) → revisar prompt/modelo y añadir el caso
al eval set de regresión.
```

Sin traza con versiones, ambos escenarios son indistinguibles: "el sistema se equivocó" sin causa asignable ni acción correctiva. La auditabilidad convierte incidentes en diagnósticos.

📊 Propiedades y comparación

Dimensión	Demo / prototipo	RAG productivo y auditable
Corpus	carpeta estática	ingesta versionada, hash y permisos por chunk
Recuperación	top-k vectorial	híbrida + re-rank + filtros ACL en el índice
Respuesta	texto libre	citas verificadas + política de rechazo
Calidad	"se ve bien"	eval set versionado en CI + muestreo en producción
Fallos	se reintentan a mano	trazas por consulta, diagnóstico por componente
Coste	ignorado	presupuesto por consulta, caché, compresión
Seguridad	implícita	inyección indirecta, ACL, procedencia del corpus
Cambios	editar y desplegar	puerta de evaluación de no-regresión

⚠️ Errores conceptuales frecuentes

1. **"Productivo = desplegado"**. Productivo significa operable: con métricas, trazas, coste conocido y camino de rollback. Un endpoint sin evals es una demo con URL.

2. **Auditar solo con logs de aplicación**. Los logs registran que algo pasó; la auditoría exige reconstruir **con qué evidencia** — chunks exactos y versión del corpus. Sin versionado del índice, la traza apunta a documentos que ya no existen.
3. **Tratar la seguridad como capa final**. El filtrado ACL después de recuperar (o peor, después de generar) ya filtró información al prompt y a las citas; el permiso se aplica en el índice.
4. **Congelar el eval set**. Un conjunto de evaluación que no incorpora los fallos reales de producción mide el sistema de hace tres meses; la traza alimenta el eval set continuamente.
5. **Optimizar métricas de componentes y no medir el conjunto**. Recall@k arriba, faithfulness arriba… y usuarios insatisfechos: el sistema se valida de punta a punta con tareas reales, no solo por piezas.

🚀 Del aprendizaje a la operación

Este proyecto es el ensayo del patrón completo, pero la operación real añade lo que ningún laboratorio reproduce: acuerdos de nivel de servicio y guardias, gestión de incidentes con usuarios afectados, cumplimiento normativo sobre los datos del corpus (retención, supresión, residencia), revisión de seguridad externa a quien construyó el sistema, gestión del cambio de modelos del proveedor (deprecaciones que obligan a re-evaluar todo) y el coste organizativo de mantener el eval set y las anotaciones humanas al día. La diferencia final no es técnica: es que alguien firma que el sistema puede responder por sus respuestas.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401 — uso: fuente primaria del mecanismo estudiado
- Gao, Y. et al. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv:2312.10997 — uso: fuente primaria del mecanismo estudiado
- Es, S. et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv:2309.15217 — uso: fuente primaria del mecanismo estudiado
- OWASP, *Top 10 for Large Language Model Applications*: https://owasp.org/www-project-top-10-for-large-language-model-applications/ — uso: marco normativo de referencia
- Greshake, K. et al. (2023). *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. arXiv:2302.12173 — uso: fuente primaria del mecanismo

estudiado
- NIST, *AI Risk Management Framework*: https://www.nist.gov/itl/ai-risk-management-framework — uso: marco normativo de referencia

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P11 · Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento	2020	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Manning, Christopher D., Raghavan, Prabhakar y Schütze, Hinrich — *Introduction to Information Retrieval*	2008	ISBN 9780521865715 · web de la obra	obra de referencia de la parte 08 · toda la parte
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	obra de referencia de la parte 08 · representaciones vectoriales de significado

Normas y documentación oficial que aplica esta clase: OWASP Top 10 for LLM Applications · AI Risk Management Framework

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: rag productivo y auditable sin usar una marca o framework como definición.
2. Explica la relación entre RAG, evals, observabilidad, seguridad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
